

IntentOS

From human intent to executable action.

IntentOS is a local, voice-native agentic execution layer. It converts natural human intent into controlled, verifiable actions over a user's applications and data. Kori provides the Hindi, Hinglish, and regional voice for the runtime.

Core Philosophy

Humans think in goals. Software forces humans to think in interfaces. IntentOS removes that translation step.

What Problem IntentOS Solves

Traditional computer interaction requires complex manual translation. The user must follow a tedious step-by-step path:

Intent -> Find application -> Understand UI ->
Navigate menus -> Locate data -> Perform action ->
Verify result

For example, a user wants to remind unpaid customers. The user opens a spreadsheet app. The user locates customer records. The user filters unpaid entries. The user opens a messaging app. The user types each reminder manually.

IntentOS transforms this process into a direct execution loop:

Intent -> Understand -> Retrieve -> Reason -> Act ->
Verify

The user states their goal directly. IntentOS handles navigation, retrieval, execution, and verification automatically.

Execution Layer vs Chatbot

IntentOS is not a chatbot. It is an execution layer between humans and software.

The LLM is not the product. The LLM acts as the intent interpreter and reasoning engine.

The IntentOS runtime manages memory structure, query planning, tool execution, and state verification.

Core Mechanism: Schema-Guided Intent Execution

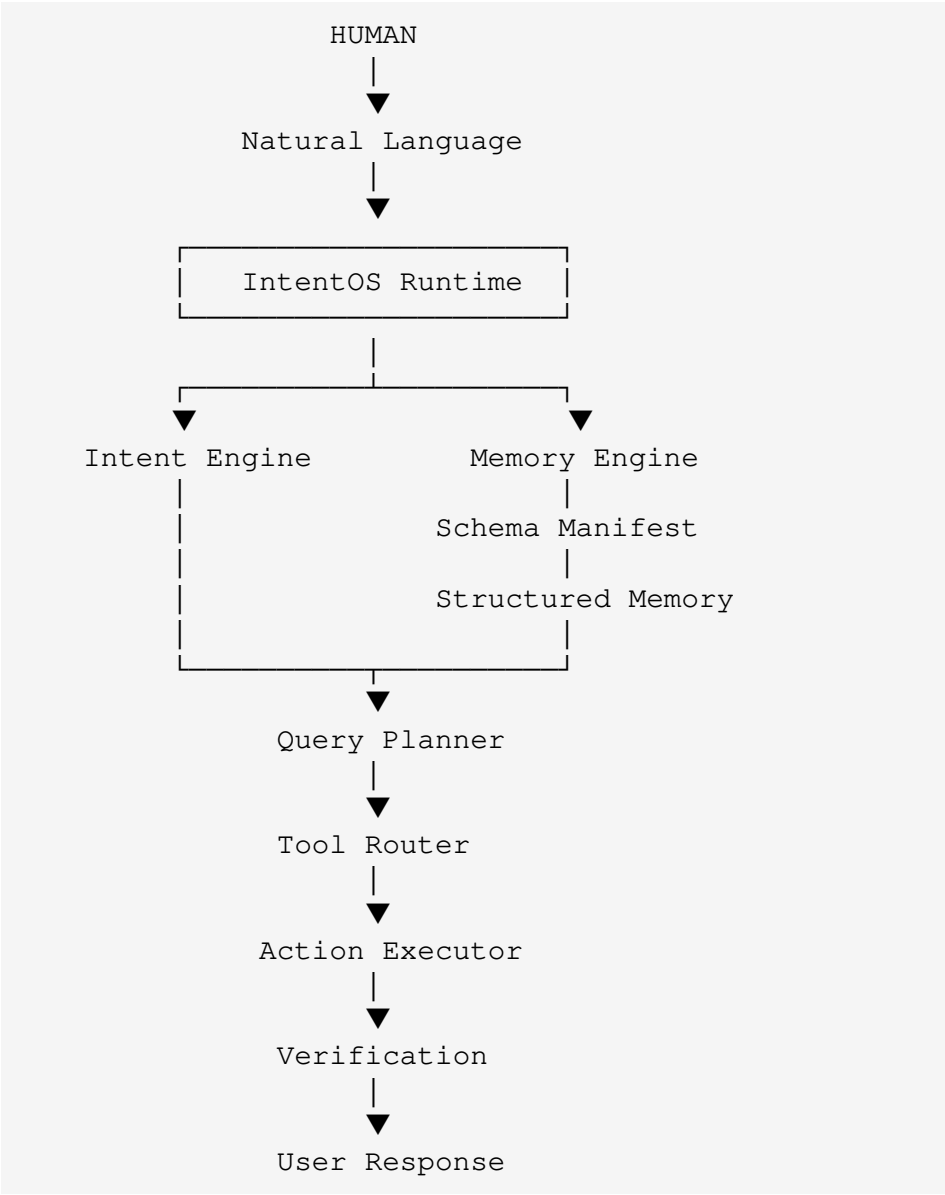
Conventional AI systems dump unstructured text or entire database tables into an LLM context. That approach is slow and unreliable.

The core mechanism of IntentOS is **Schema-Guided Intent Execution**.

1. IntentOS maintains a machine-readable `schema_manifest`.
2. The manifest defines entities, table relationships, column purposes, and allowed tool operations.
3. The system understands the structure of its own memory.
4. The Query Planner inspects the manifest when an intent arrives.
5. It generates a targeted query to extract only the minimal relevant records.
6. Validated tools execute the required actions.
7. The runtime verifies the post-action state against expected outcomes.

Core Principle: Do not give the AI more memory. Give it better access to structured memory.

Core Architecture



The 5-Step Core Loop

Remember -> Retrieve -> Reason -> Act -> Verify

- 1. **Remember:** Maintain persistent, structured local memory across sessions.
- 2. **Retrieve:** Extract relevant data using schema-guided queries.
- 3. **Reason:** Build execution plans from structured intent representations.
- 4. **Act:** Execute operations through registered, validated tools.
- 5. **Verify:** Compare observed state changes against expected parameters.

Risk-Aware Local Memory

IntentOS stores memory locally in an SQLite database. Memory is classified by risk:

Memory Type	Behavior
context	Saved automatically for active conversation context.
preference	Saved automatically for user defaults like language or reminder time.
personal	Requires explicit user confirmation before saving.
secret	Rejected completely. Passwords, API keys, tokens, and private keys are blocked.

Secret-like keys such as passwords, tokens, API keys, and private keys are strictly blocked even if mislabeled.

Action Execution & Safety Boundaries

IntentOS separates operations by impact level.

- Read operations execute automatically.
- Low-risk write operations execute automatically and run state verification.
- High-impact write operations require explicit user confirmation before execution.

Every plan step is validated against registered tools. Unregistered tools are blocked.

Database Schema

The local SQLite database (`database.py`) uses a domain-neutral schema:

- `profiles`: User profiles, language settings, and time zones.
- `entities`: Structured personal memory records.
- `entities.memory_class`: Memory classification categories.
- `entities.consent_status`: Confirmation status for personal memory.
- `conversation_sessions` & `conversation_messages`: Session tracking and message logs.
- `intents`: Structured intent models.
- `plans` & `plan_steps`: Executable plan graphs.
- `tool_executions`: Tool call logs and execution records.
- `verifications`: Verification results.
- `tasks`: Reminders and task state.
- `preferences`: System and user preferences.
- `events`: Audit trail event log.
- `schema_manifest`: Machine-readable entity and tool definitions.

Local-First & Snapdragon Positioning

IntentOS is built for local execution. It reduces cloud dependency and protects data privacy.

The runtime architecture is designed for Snapdragon X-series PCs and NPU acceleration.

MVP Status & Submission Honesty

Snapdragon NPU acceleration is an architectural target for production deployment.

The MVP validates core runtime mechanisms, schema-guided retrieval, tool routing, and verification pipelines.

VyapaarSaathi business entities are handled via domain adapters.

Quick Start

Requirements

Python 3.12 or newer.

Setup Commands

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m py_compile database.py tools.py prompts.py agent.py
```

Set environment variables in `.env` based on `.env.sample`.

Launch Agent

```
python agent.py dev
```

Hackathon Evaluation Alignment

Technical Implementation

- SQLite structured local memory.
- Machine-readable schema manifest engine.
- Registered tool validation and registry.
- Execution planning and state verification.

Application Use Case & Innovation

- Replaces manual software UI navigation with voice intent.
- Replaces generic text RAG with Schema-Guided Intent Execution.
- Implements risk-aware memory classification.

Deployment & Accessibility

- Voice-native interface powered by Kori.
- Supports Hindi, Hinglish, and regional languages.
- Designed for local Snapdragon edge deployment.

Presentation & Documentation

- Fully documented architecture and database schema.
- Clear separation between runtime core and domain adapters.
- Complete submission honesty regarding cloud vs local inference.